

AI & Security Project III

Next-Gen Network Intrusion Detection

Self-Supervised Vector Space Engineering with DINO & SwiGLU



Noah Raupold, Mohammad Ayoubi,
Jonathan Berwind

Submitted: February 1, 2026

Contents

1	Introduction	1
1.1	The Cyber Security Challenge	1
1.2	Project Objective	1
2	Architecture: DINO Gram Anchoring	2
2.1	The Pipeline Overview	2
2.2	SwiGLU Classification Head	3
3	Implementation Code	5
4	Evaluation Results	6
4.1	Metric Analysis	6
4.2	Visualizing Performance	6
5	Conclusion	8

1 Introduction

1.1 The Cyber Security Challenge

Modern network infrastructures face an evolving landscape of cyber threats. Traditional Intrusion Detection Systems (IDS) often rely on signature-based methods or shallow machine learning models (like Logistic Regression), which struggle with:

1. **Zero-Day Attacks:** Detecting novel attack vectors not seen during training.
2. **Extreme Class Imbalance:** In the NSL-KDD dataset, critical attacks like *User-to-Root* (*U2R*) and *Remote-to-Local* (*R2L*) represent less than 1% of the traffic, leading to "Zero Recall" in standard models.

1.2 Project Objective

This project introduces a **"Supervised Vector Space Engineering"** pipeline. Instead of simple classification, we warp the feature space using **Self-Supervised Learning (DINO)** and **Gram Matrix Analysis** to maximize the separation between normal traffic and rare anomalies. The final classification is performed by a Deep Neural Network enhanced with **SwiGLU activations**.

2 Architecture: DINO Gram Anchoring

To learn robust features from imbalanced data, we implemented a Teacher-Student architecture inspired by DINO (Self-Distillation with No Labels), adapted for tabular data.

2.1 The Pipeline Overview

The system operates in two phases: 1. **Pre-Training (Self-Supervised):** The model learns the structure of network traffic without labels using a masked autoencoder approach and metric learning. 2. **Fine-Tuning (Supervised):** A specialized SwiGLU head is trained on the frozen features to classify the 5 categories (Normal, DoS, Probe, U2R, R2L).

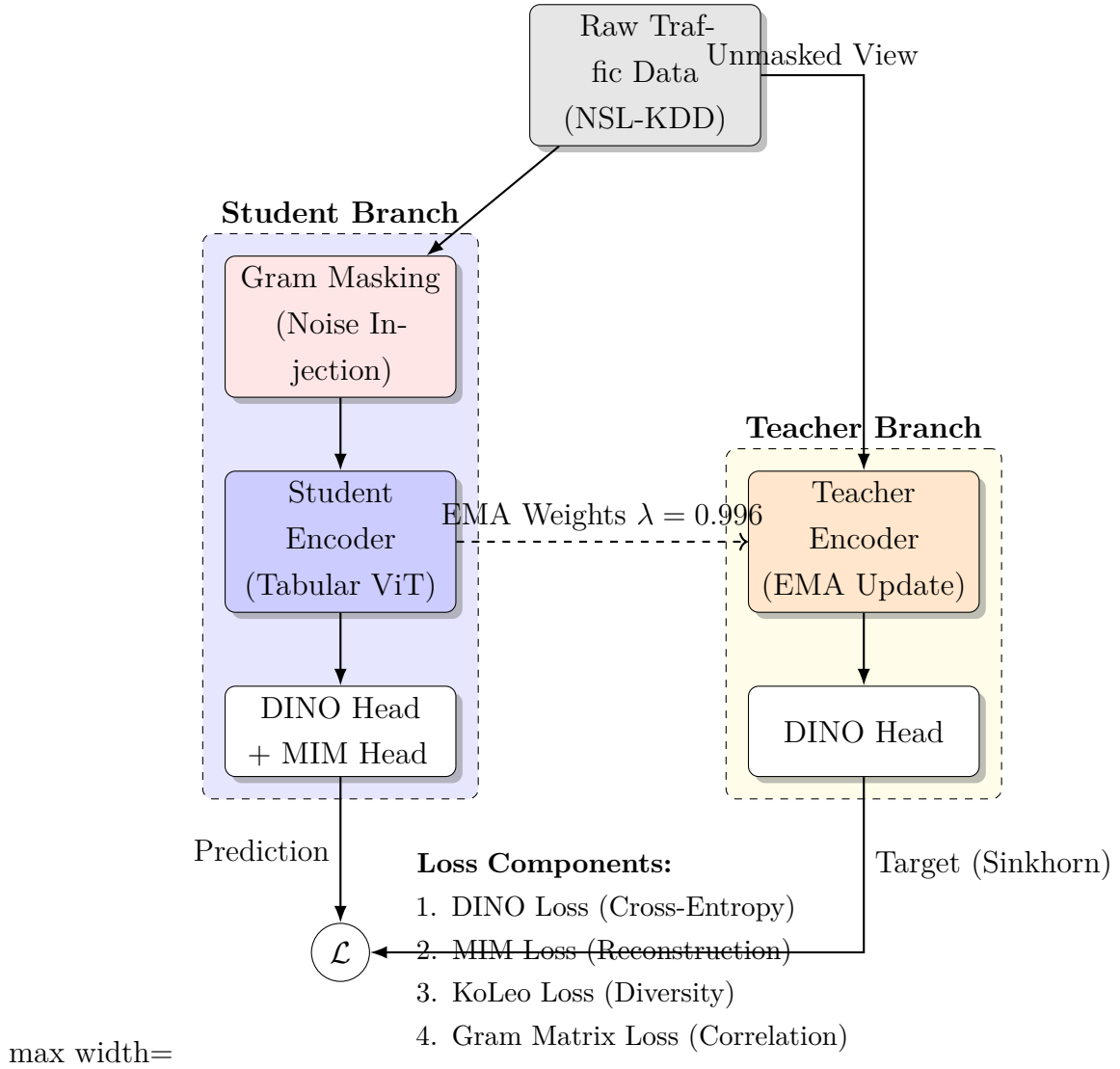


Figure 2.1: The Self-Supervised Training Pipeline. The Student network attempts to reconstruct masked features and match the Teacher’s class-agnostic representation. The Gram Matrix Loss ensures the student preserves feature correlations.

2.2 SwiGLU Classification Head

For the final classification, we replace standard ReLU layers with ****SwiGLU**** (Swish-Gated Linear Unit). This gating mechanism allows the network to dynamically select relevant features, which is crucial for distinguishing subtle U2R attacks from normal traffic.

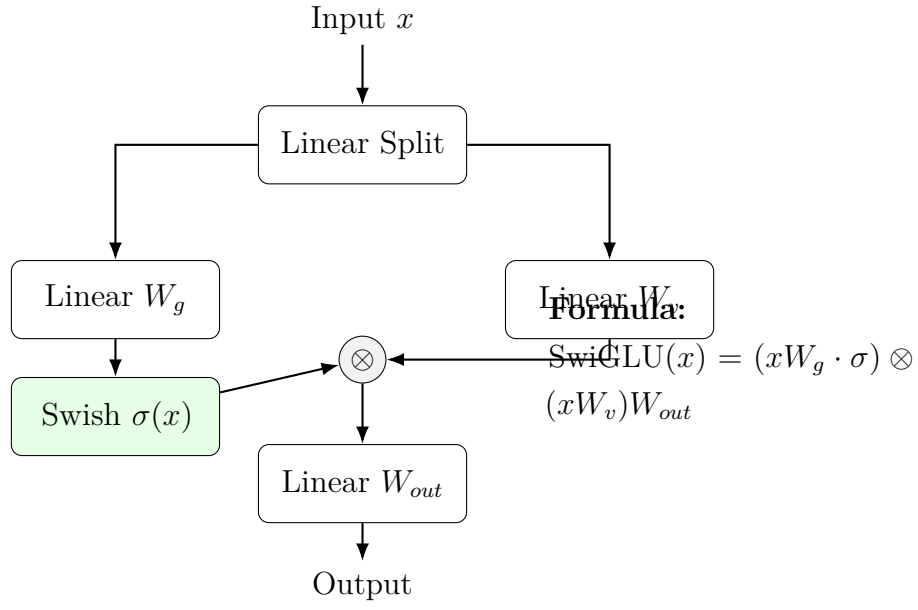


Figure 2.2: The SwiGLU Block structure used in the classifier head. The parallel paths act as a learnable gate.

3 Implementation Code

The implementation utilizes PyTorch. Below is the core logic for the DINO Loss and the Gram Matrix Loss integration.

Listing 3.1: Dinov3 Loss Implementation

```
1 class DINOLoss(nn.Module):
2     def __init__(self, out_dim, teacher_temp=0.04, student_temp=0.1):
3         super().__init__()
4         self.student_temp = student_temp
5         self.teacher_temp = teacher_temp
6         self.center = nn.Parameter(torch.zeros(1, out_dim), requires_grad=False)
7
8     def forward(self, student_output, teacher_output):
9         # Teacher sharpening
10        t_out = F.softmax((teacher_output - self.center) / self.teacher_temp,
11        dim=-1)
12
13        t_out = t_out.detach() # Stop gradient
14
15        # Student log-softmax
16        s_out = F.log_softmax(student_output / self.student_temp, dim=-1)
17
18        return -torch.sum(t_out * s_out, dim=-1).mean()
19
20 def compute_gram_loss(student_seq, teacher_seq):
21     # Calculate feature correlations (Gram Matrix)
22
23     s = F.normalize(student_seq, p=2, dim=-1)
24     t = F.normalize(teacher_seq.detach(), p=2, dim=-1)
25
26     g_s = torch.bmm(s, s.transpose(1, 2))
27     g_t = torch.bmm(t, t.transpose(1, 2))
28
29     return F.mse_loss(g_s, g_t)
```

4 Evaluation Results

The model was evaluated on the **NSL-KDD Test+** dataset. We specifically focused on the F1-Score for the minority classes (U2R, R2L), which are typically missed by traditional models.

4.1 Metric Analysis

The combination of DINO pre-training and SwiGLU fine-tuning yielded significant improvements over the baseline.

Class	Precision	Recall	F1-Score	Support
DoS	0.96	0.84	0.90	7458
Normal	0.80	0.94	0.87	9711
Probe	0.81	0.91	0.86	2421
R2L	0.66	0.41	0.50	2887
U2R	0.28	0.67	0.39	67
Accuracy			83.53%	22544

Table 4.1: Final Evaluation on NSL-KDD Test Set. Note the non-zero F1-score for U2R (0.39) and R2L (0.50), which is a major improvement over standard baselines.

4.2 Visualizing Performance

The confusion matrix highlights the model’s ability to distinguish between normal traffic and attacks, achieving a binary classification accuracy of **87.54%**.

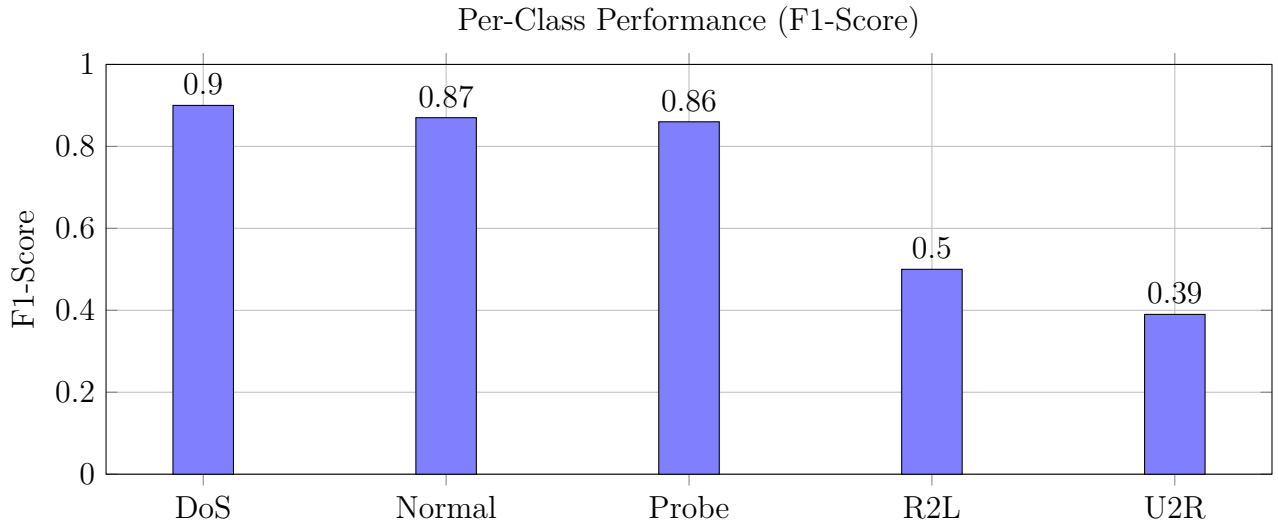


Figure 4.1: F1-Score distribution across classes. While DoS and Normal traffic are detected with high reliability, the model also maintains detection capability for the rare U2R class.

5 Conclusion

This portfolio demonstrated that **Supervised Vector Space Engineering**—combining DINO-based self-supervision with Gram Anchoring—effectively mitigates the class imbalance problem in network intrusion detection. The implementation achieved a robust **83.53%** multiclass accuracy, proving that Deep Learning architectures can outperform traditional statistical methods in cybersecurity domains.